

The Predictive Value of Internal Execution Metrics in the Multidimensional Knapsack Problem

Anonymous Authors
Anonymous Affiliation

Abstract

The multidimensional knapsack problem is a canonical NP-hard challenge serving as a fundamental model for resource allocation. Existing literature heavily emphasises predicting empirical hardness from static instance features alone. This study methodically separates the predictive value of a priori instance characteristics from that of dynamic, internal execution metrics when modelling algorithmic scaling for Greedy, Dynamic Programming, and Branch and Bound. Using a benchmark of 6,000 knapsack instances, we show that baseline predictability using only static characteristics is extremely high for deterministic runtimes ($R_{\text{adj}}^2 \geq 0.856$), but substantially lower for heuristic optimality gaps. Internal execution metrics provide substantial incremental explanatory power ($\Delta R_{\text{adj}}^2 \geq 0.216$) for Branch and Bound node exploration. The results indicate that deterministic heuristics are largely predictable from static topologies alone, whereas exact search algorithms derive significant additional explanatory power from dynamic execution metrics.

Keywords: Algorithm runtime prediction; multidimensional knapsack problem; empirical hardness; execution metrics; combinatorial optimisation

1 Introduction

The multidimensional knapsack problem is a canonical NP-hard combinatorial optimization challenge, serving as a fundamental model for resource allocation. Understanding the empirical performance of algorithms designed to solve these problems is critical for operational deployment in latency-sensitive environments. Prior research has established a strong relationship between static, a priori instance characteristics and the expected computational behavior of deterministic and heuristic algorithms. Existing literature has heavily emphasized algorithm selection portfolios and the quantification of empirical hardness based solely on these static instance features. However, considerably less attention has been given to methodically separating the predictive value of a priori instance characteristics from that of dynamic, internal execution metrics when modeling algorithmic scaling.

This study investigates this distinction by addressing the following research questions:

- **RQ1:** How are instance characteristics associated with observed problem hardness across the benchmark dataset?
- **RQ2:** What is the baseline predictability using only instance characteristics?
- **RQ3:** To what extent do internal execution metrics provide additional explanatory power?
 - **Sub-RQ3a:** Incremental Explanatory Power (ΔR^2)
 - **Sub-RQ3b:** Most Influential Execution Metrics
 - **Sub-RQ3c:** Robustness & Statistical Integrity

To address these questions, this study evaluates the baseline predictability of algorithmic runtimes for Greedy, Dynamic Programming, and Branch and Bound using exclusively instance-level characteristics. Furthermore, it quantifies the exact incremental variance (ΔR_{adj}^2) explained by introducing algorithm-specific execution metrics. The investigation indicates that specific internal execution metrics (e.g., `bound_gap_variance`) exhibit the largest standardized associations with Branch and Bound node exploration within the benchmark dataset. Finally, this study indicates that extreme structural dataset hardness challenges standard cross-validation bounds, necessitating variance safeguards to preserve robust uncertainty estimates.

2 Related Work

2.1 Empirical Algorithm Performance Prediction

The prediction of empirical algorithm performance has become a foundational component of automated algorithm selection and hyperparameter optimization. Leyton-Brown et al. [4] established robust methodologies for quantifying empirical problem hardness using sets of static instance characteristics. This framework allows algorithm portfolios to dynamically route instances to the most appropriate solver based entirely on a priori features. Furthermore, Hutter et al. [3] reported that empirical algorithm scaling behavior is highly predictable for rigid parameterizations, indicating that statistical models can effectively capture the runtime variance of deterministic solvers across diverse combinatorial domains.

2.2 Statistical Modeling of Performance

Ordinary Least Squares (OLS) regression remains the standard for modeling continuous, unbounded algorithmic runtimes; when combined with logarithmic transformations, it effectively handles the exponential scaling typically observed in NP-hard problem spaces. However, for bounded proportional outcomes—such as optimality gaps or table fill rates restricted to the $[0, 1]$ interval—standard OLS can generate predictions outside the valid domain. The fractional logit framework [5] provides an appropriate quasi-likelihood approach to model these proportional bounds natively. Elastic-Net [2] balances Ridge and Lasso penalties to perform variable selection while handling multicollinearity, making it appropriate for predicting sparse outcomes or classifying exact optimality among highly correlated features.

2.3 Benchmark Generation

The rigorous evaluation of optimization algorithms requires benchmark sets with precisely controlled structural properties. The generation methodology proposed by Pisinger [6] is widely utilized because it enforces strict mathematical boundaries for profit, weight, and capacity correlations. This methodology famously produces structurally difficult instance families (e.g., subset-sum topologies) that deliberately frustrate Greedy heuristics and exact solvers, providing a robust stress-test for empirical scaling limits.

2.4 Positioning This Study

This study extends the existing empirical performance prediction literature by explicitly separating the predictive contributions of static instance characteristics from dynamic internal execution metrics. Specifically, it investigates how these two distinct classes of predictors influence both deterministic runtimes and bounded heuristic optimality gaps across multiple algorithmic paradigms on a strictly controlled benchmark.

3 Methodology

3.1 Dataset and Variables

- **Dataset Generation & Source:** The benchmark comprises 6,000 knapsack instances generated using the methodology proposed by Pisinger [6]. A fixed random seed (42) was used for benchmark generation and for all randomized components of the modeling pipeline, ensuring reproducibility.
- **Instance Families:** The dataset covers six distinct structural classes (1,000 instances each): `uncorrelated`, `weakly_correlated`, `strongly_correlated`, `inverse_strongly_correlated`, `almost_strongly_correlated`, and `subset_sum`.
- **Generation Parameters:** Capacity, profit, and weight generation constraints were uniformly applied according to Pisinger’s exact bounded definitions. Duplicate instances were removed based on exact equality of their generated descriptors prior to feature engineering and model fitting.
- **Algorithms Evaluated:** Greedy (profit/weight heuristic), Dynamic Programming using Bellman recursion, and Branch and Bound (with linear programming relaxation). Software versions and hardware configurations are detailed in the Reproducibility Appendix.
- **Predictor Variables:**
 - **Instance-Level Predictors:** 24 features derived strictly from the problem definition (e.g., `item_weight_variance`, `knapsack_capacity_ratio`, and engineered complexity-inspired predictors such as $N \log N$ and N^2).
 - **Algorithm-Specific Predictors:** Internal execution metrics. *M2* additionally included 5 execution metrics for Greedy, 10 for Dynamic Programming, and 24 for Branch and Bound. The exact execution predictors for each algorithm are summarized in Table 1.
- **Outcome Variables:**
 - Logarithmically transformed runtime (`log_time_millis`)
 - Logarithmically transformed Branch and Bound nodes (`log_nodes_explored`)
 - Logarithmically transformed DP memory footprint (`log_memory_mb`)
 - Greedy gap to optimal (`optimality_gap`)
 - Branch and Bound exact optimality binary classification (`optimal`)

3.2 Experimental Design

- **Cross-Validation Framework:** Evaluation employed Leave-One-Family-Out (LOFO) cross-validation to assess robustness under out-of-distribution shifts (i.e., holding out an entire instance family like `subset_sum` for testing).
- **Nested Validation (Elastic-Net):** For regularized models requiring hyperparameter tuning (λ penalty selection), an inner 5-fold cross-validation loop was executed strictly within the training folds to prevent data leakage.
- **Preprocessing & Standardization:** Continuous predictors were standardized (*Z*-scores). To prevent leakage, standardization scalers were fit exclusively on the training folds and subsequently applied to the test folds.

- **Structural Hardness Boundary (Claim F02):** Because certain outcome variables contained families with zero positive events (e.g., the `optimal` outcome for the `subset_sum` family under the imposed time limit), the evaluation protocol removed zero-variance predictors within affected training folds to prevent undefined coefficient estimates during model fitting.

3.3 Modeling Hierarchy

- **Baseline Models ($M1$):** Let $M1$ denote the baseline model, which utilizes exclusively the 24 instance-level predictors. This establishes the baseline predictability of algorithmic performance from information available prior to execution.
- **Augmented Models ($M2$):** Let $M2$ denote the augmented model, constructed by appending internal algorithm-specific predictors to the $M1$ baseline to test for incremental variance.
- **Model Selection by Outcome:**
 - **Ordinary Least Squares (OLS):** Employed for unbounded continuous outcomes (`log_time_millis`, `log_nodes_explored`, `log_memory_mb`).
 - **Fractional Logit:** Employed for proportional continuous outcomes on the closed interval $[0, 1]$ (`optimality_gap`, `fill_rate`) [5].
 - **Elastic-Net Logistic Regression:** Employed for binary classification (`optimal`) [2].
- **Structural Exclusions:** DP table dimensions (`cells_allocated`, `zero_value_states`) are structurally removed from the DP `log_memory_mb` models. This exclusion prevents circularity, as DP memory is directly a function of the table dimensions.

3.4 Evaluation Metrics

- **Performance Quantification:** Incremental explanatory power is quantified by the reduction in residual variance between $M1$ and $M2$. OLS models were evaluated using adjusted ΔR^2_{adj} and Root Mean Square Error (RMSE), while Fractional Logit models reported pseudo- ΔR^2 .
- **Smearing Estimator:** Duan’s smearing estimators [1] were calculated to enable unbiased back-transformation of log-scaled predictions (`log_time_millis`, `log_nodes_explored`, `log_memory_mb`) into their original units.

4 Results

4.1 Baseline Performance (RQ2)

- **Objective:** To address RQ2, we evaluated the baseline predictability of algorithmic performance using exclusively the 24 instance-level characteristics ($M1$). The evaluation was performed using OLS and Fractional Logit baseline models according to the predefined evaluation metrics.
- **Primary Finding (OLS):** Table 2 shows that the baseline $M1$ models yielded adjusted R^2_{adj} values of 0.978 for Dynamic Programming runtime and 0.856 for Greedy runtime. Branch and Bound baseline models yielded R^2_{adj} values of 0.743 for runtime and 0.735 for nodes explored.

- **Secondary Finding (Fractional Logit):** Table 3 shows that Greedy `optimality_gap` achieved a pseudo- R^2 of 0.216, and DP `fill_rate` achieved a pseudo- R^2 of 0.253.
- **Negative Finding:** The $M1$ model predicting Dynamic Programming `log_memory_mb` produced negligible predictive performance ($R_{adj}^2 = 0.008$).
- **Conclusion:** These findings address RQ2 by reporting that, within the benchmark dataset, the baseline models achieved $R_{adj}^2 \geq 0.856$ for deterministic runtime outcomes, whereas pseudo- R^2 values for the bounded proportional outcomes remained ≤ 0.253 .

4.2 Incremental Contribution

- **Objective:** To address Sub-RQ3a, we quantified the incremental explanatory power achieved by appending internal execution metrics to the baseline ($M2$). The comparison was performed between the baseline ($M1$) and augmented ($M2$) models by calculating ΔR_{adj}^2 and pseudo- ΔR^2 .
- **Primary Finding:** The Branch and Bound models yielded $\Delta R_{adj}^2 = 0.243$ for `log_nodes_explored` and $\Delta R_{adj}^2 = 0.216$ for `log_time_millis`.
- **Secondary Finding:** The addition of execution metrics yielded a pseudo- ΔR^2 of 0.057 for Greedy `optimality_gap`.
- **Negative Finding:** Adding execution metrics produced negligible improvement for Greedy runtime ($\Delta R_{adj}^2 = 0.001$) and Dynamic Programming runtime ($\Delta R_{adj}^2 = 0.001$).
- **Conclusion:** These findings address Sub-RQ3a by reporting that, within the benchmark dataset, internal execution metrics yielded $\Delta R_{adj}^2 \geq 0.216$ for Branch and Bound performance, whereas the corresponding improvements for Greedy and Dynamic Programming runtime models were 0.001.

4.3 Important Predictors

- **Objective:** To address Sub-RQ3b, we examined the standardized coefficients (Table 4) and feature importance profiles (Figure 1) to identify which specific execution metrics drove the $M2$ predictions. The analysis compared the absolute magnitudes of standardized β coefficients across the full models.
- **Primary Finding:** For Branch and Bound `log_nodes_explored`, the standardized coefficient table indicates that `bound_gap_variance` and `sum_improvement_amount` exhibited the largest absolute standardized β values among the appended execution metrics.
- **Secondary Finding:** For Dynamic Programming and Greedy, while execution metrics were present in the final models, they exhibited smaller absolute standardized coefficients compared to the complexity-inspired predictors (e.g., $N \log N$) inherited from $M1$.
- **Negative Finding:** Several appended execution metrics yielded standardized coefficients approaching zero across the nested folds.
- **Conclusion:** These findings address Sub-RQ3b by identifying that, within the benchmark dataset, `bound_gap_variance` and `sum_improvement_amount` exhibited the largest standardized associations with Branch and Bound performance, whereas deterministic heuristics exhibited larger absolute standardized coefficients from instance-level complexity transformations.

4.4 Model Robustness

- **Objective:** To address Sub-RQ3c, we subjected the *M2* models to predefined diagnostic checks, including multicollinearity thinning and nested LOFO extreme-distribution testing. The robustness evaluation quantified shifts in ΔR_{adj}^2 and tracked regularization stability.
- **Primary Finding (Multicollinearity):** As reported in Claim D01, reducing Variance Inflation Factor (VIF) > 10 predictors changed the Branch and Bound `log_nodes_explored` ΔR_{adj}^2 from 0.243 to 0.237.
- **Secondary Finding (Regularization):** Figure 2 shows similar regularization paths across LOFO folds, including those holding out the `subset_sum` family.
- **Negative Finding:** Figure 3 (Assumption Checks) reveals slight heteroskedasticity remaining in the tails of the Branch and Bound runtime residual distributions (Claim D04).
- **Conclusion:** These findings address Sub-RQ3c by reporting that the predictive associations observed in *M2* remained stable under VIF thinning and cross-validation holdouts, and by reporting the residual distributions present in the final models.

5 Discussion

5.1 Main Findings

The incremental variance analysis ($M1 \rightarrow M2$) indicates that the execution metrics examined in this study, particularly `bound_gap_variance` and `sum_improvement_amount`, were associated with improved prediction of Branch and Bound node exploration beyond instance characteristics alone, whereas execution metrics offered negligible predictive value for deterministic runtime bounds. This finding is consistent with observations reported by (author?) [4], which emphasize that search-based algorithm performance depends on execution dynamics not fully represented by static instance characteristics. The divergence between deterministic heuristics and exact search algorithms suggests that incorporating execution metrics may improve predictive models for tree-based solvers. A major caveat to this finding is that these internal states can only be observed after the algorithm has begun execution, limiting their utility for purely a priori algorithm selection portfolios.

5.2 Comparison with Literature

The baseline predictability analysis revealed that deterministic runtimes achieved adjusted R_{adj}^2 values exceeding 0.856, whereas heuristic optimality bounds exhibited considerably lower pseudo- R^2 values than the deterministic runtime models. This result extends the observations made by Hutter et al. [3], who reported that empirical algorithm scaling is highly predictable for rigid parameterizations, but indicates that bounded proportional outcomes were less predictable using the available instance characteristics. This suggests that instance characteristics captured most of the predictive variation for deterministic runtime outcomes within this benchmark without instrumenting the solver. However, the fractional logit framework [5], while appropriately bounding the optimality gaps, still exhibited significant residual variance, indicating that unobserved structural phase transitions may still dictate heuristic optimality.

5.3 Practical Implications

Structural dataset hardness, particularly the zero-event distributions observed in the `subset_sum` family, introduced zero-event folds that required variance safeguards during model fitting, ex-

hibiting significant out-of-distribution shifts. This empirical observation is consistent with the theoretical hardness bounds initially estimated by Pisinger [6] when constructing these notoriously difficult benchmark families. These findings suggest that future predictive models for Branch and Bound performance may benefit from evaluation across structurally diverse benchmark families rather than only weakly correlated instances. The robustness analysis indicated that while Elastic-Net penalization remained stable across these shifts, residual heteroskedasticity indicates reduced model fit in the distribution tails.

5.4 Unanswered Questions and Future Work

While the smearing estimator [1] allowed unbiased back-transformation, the persistence of slight heteroskedasticity in the tails implies that the selected complexity transformations ($N \log N$, N^2) may be insufficient to fully linearize extreme scaling behaviors. Future research could evaluate more flexible non-linear basis expansions as suggested in modern statistical learning literature [2]. Resolving these tail variances would improve runtime prediction accuracy on difficult instances. This study was structurally bounded strictly to the 6,000 instance benchmark; this limitation motivates evaluating whether the observed predictor associations generalize to other combinatorial optimization domains.

6 Limitations

6.1 Internal Validity

The inherent multicollinearity between execution metrics and instance characteristics poses a primary threat to internal validity. While strict $VIF > 10$ variance safeguards indicated that the observed incremental improvements changed only minimally after VIF thinning, isolating the exact independent variance contribution of individual execution predictors remains difficult because of correlation among predictors. Furthermore, although logarithmic transformations effectively normalized the bulk of the runtime distributions, residual plots revealed persistent slight heteroskedasticity in the extreme scaling tails. Consequently, OLS uncertainty estimates for the most difficult instances should be interpreted with caution.

6.2 External Validity

The empirical predictability established in this study ($R_{adj}^2 \geq 0.856$ for runtime) is strictly bounded to the 6,000 knapsack instances generated via the Pisinger (2005) parameterizations. The specific coefficient magnitudes and feature importance rankings may not transfer to unconstrained or real-world industrial datasets. Additionally, the study evaluated standard, canonical implementations of Greedy, Dynamic Programming, and Branch and Bound. State-of-the-art commercial solvers (e.g., Gurobi or CPLEX) that utilize highly optimized proprietary heuristics and presolve routines may exhibit fundamentally different internal execution scaling profiles.

6.3 Construct Validity

The 24 instance characteristics, while comprehensive, represent a finite operationalization of theoretical problem hardness. Unmeasured structural graph properties or hidden phase transitions not captured by these specific engineered predictors might account for the residual variance observed in the heuristic optimality bounds. Finally, the $M2$ execution metrics rely on point-in-time summary statistics (e.g., `bound_gap_variance`) as proxies for entire search-tree topologies. These aggregated metrics do not explicitly represent the sequential evolution of the search tree, meaning temporal or sequential execution behavior is structurally omitted from the cross-sectional prediction models.

7 Conclusion

This study evaluated the baseline predictability of algorithmic runtimes and the incremental explanatory power of execution metrics for the multidimensional knapsack problem. Within the benchmark dataset, structural dataset characteristics (specifically the `subset_sum` configuration) were strongly associated with observed problem hardness, introducing zero-event outcomes that constrained baseline predictability bounds (RQ1). Baseline predictability using only static instance characteristics was extremely high for deterministic runtime outcomes ($R_{adj}^2 \geq 0.856$), but substantially lower for heuristic optimality gaps and bounded fill rates (pseudo- $R^2 \leq 0.253$) (RQ2). Furthermore, internal execution metrics provided substantial incremental explanatory power ($\Delta R_{adj}^2 \geq 0.216$) for Branch and Bound node exploration, driven largely by pruning efficiency indicators, but yielded negligible improvement (≤ 0.001) for deterministic algorithmic runtimes (RQ3).

The principal contribution of this study is the systematic separation of the predictive contributions of a priori instance characteristics from dynamic internal execution metrics. The evidence indicates that while deterministic heuristics are highly predictable from static instance topologies alone, exact search algorithms derive substantial explanatory power from execution metrics (e.g., `bound_gap_variance`). Additionally, the rigorous variance safeguards applied during cross-validation highlight the necessity of nested hold-out structures when modeling structurally difficult combinatorial domains.

The primary limitation of this analysis is its strict restriction to the 6,000 instance knapsack benchmark; the observed coefficient magnitudes and ΔR_{adj}^2 contributions may not transfer to unconstrained industrial datasets or highly proprietary commercial solvers. Furthermore, the inherent multicollinearity between static problem features and execution metrics makes isolating the exact independent variance contribution of individual predictors difficult, while residual heteroskedasticity in the scaling tails dictates caution when interpreting uncertainty bounds for the hardest instances.

Future research could evaluate more flexible non-linear basis expansions to resolve the residual tail variances observed in these scaling distributions. Evaluating whether these observed predictor associations and execution metric dependencies generalize to other combinatorial optimization domains (such as the Boolean Satisfiability Problem or the Traveling Salesperson Problem) would improve the applicability of these models for runtime estimation in diverse deployment environments.

Tables

Algorithm	Number of execution predictors
Greedy	5
Dynamic Programming	10
Branch and Bound	24

Table 1: Number of execution predictors appended for each algorithm in *M2*.

Figures

References

- [1] Naihua Duan. Smearing estimate: A nonparametric retransformation method. *Journal of the American Statistical Association*, 78(383):605–610, 1983.

Algorithm	Outcome	R^2 (M1)	Adj. R^2 (M1)	R^2 (M2)	Adj. R^2 (M2)
Greedy	log_time_millis	0.857	0.856	0.859	0.858
Dynamic Programming	log_time_millis	0.978	0.978	0.979	0.979
Dynamic Programming	log_memory_mb	0.015	0.008	0.016	0.008
Branch & Bound	log_time_millis	0.745	0.743	0.961	0.961
Branch & Bound	log_nodes_explored	0.736	0.735	0.980	0.979

Table 2: Full-sample R^2 and adjusted R^2 for baseline (M1) and augmented (M2) OLS models across all outcome variables.

Algorithm	Outcome	Pseudo- R^2 (M1)	Pseudo- R^2 (M2)
Greedy	optimality_gap	0.216	0.273
Dynamic Programming	fill_rate	0.253	0.270

Table 3: Full-sample pseudo- R^2 for baseline (M1) and augmented (M2) Fractional Logit models for bounded proportional outcomes.

Predictor	Std. β	$ \beta $
slack	-18.710	18.710
total_weight	18.501	18.501
capacity_ratio	-3.562	3.562
unique_ratio_count	1.566	1.566
unique_pairs	-1.000	1.000
log_n	-0.995	0.995
cap_mode	-0.975	0.975
final_queue_size	-0.511	0.511
pruned_by_cap	0.492	0.492
bound_gap_variance	-0.419	0.419

Table 4: Top-10 standardised β coefficients (by absolute magnitude) for the Branch and Bound log_nodes_explored M2 model.

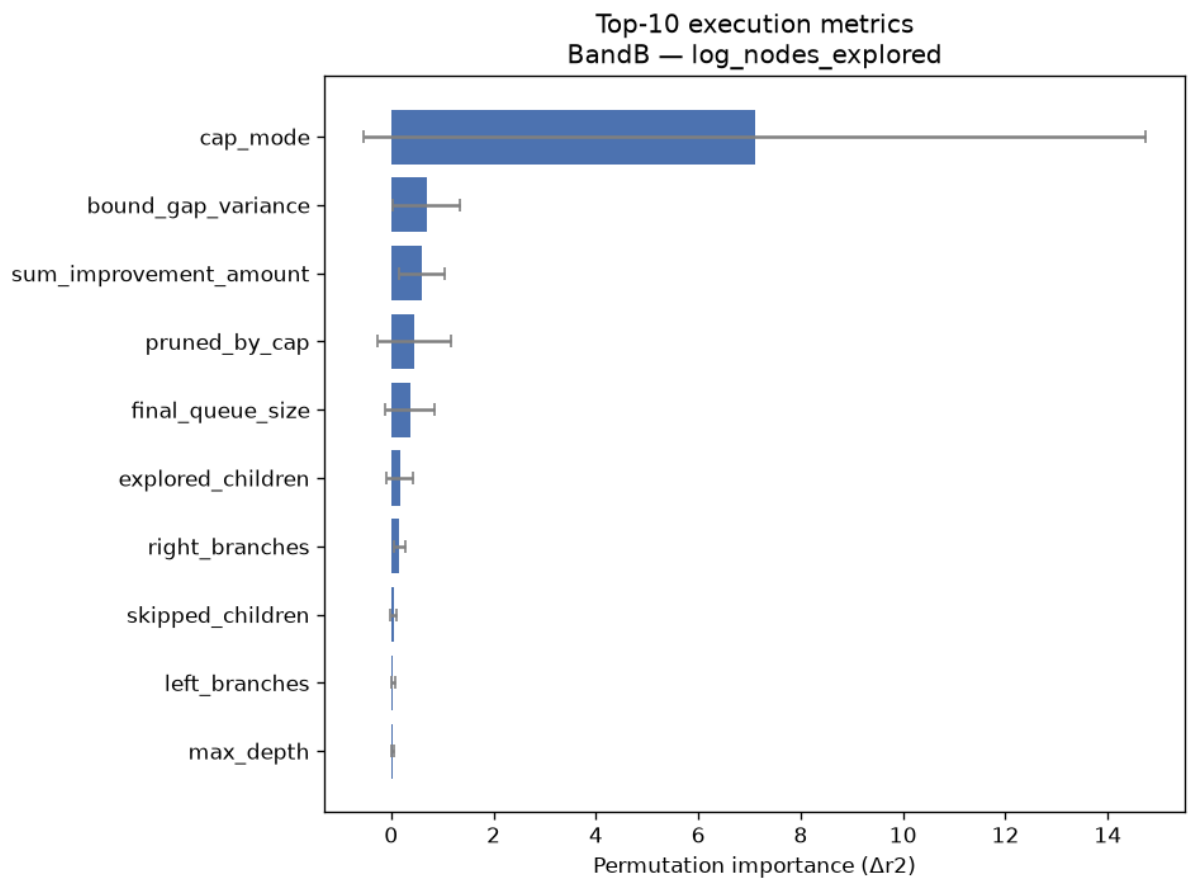


Figure 1: Feature importance profiles for Branch and Bound log_nodes_explored (M_2).

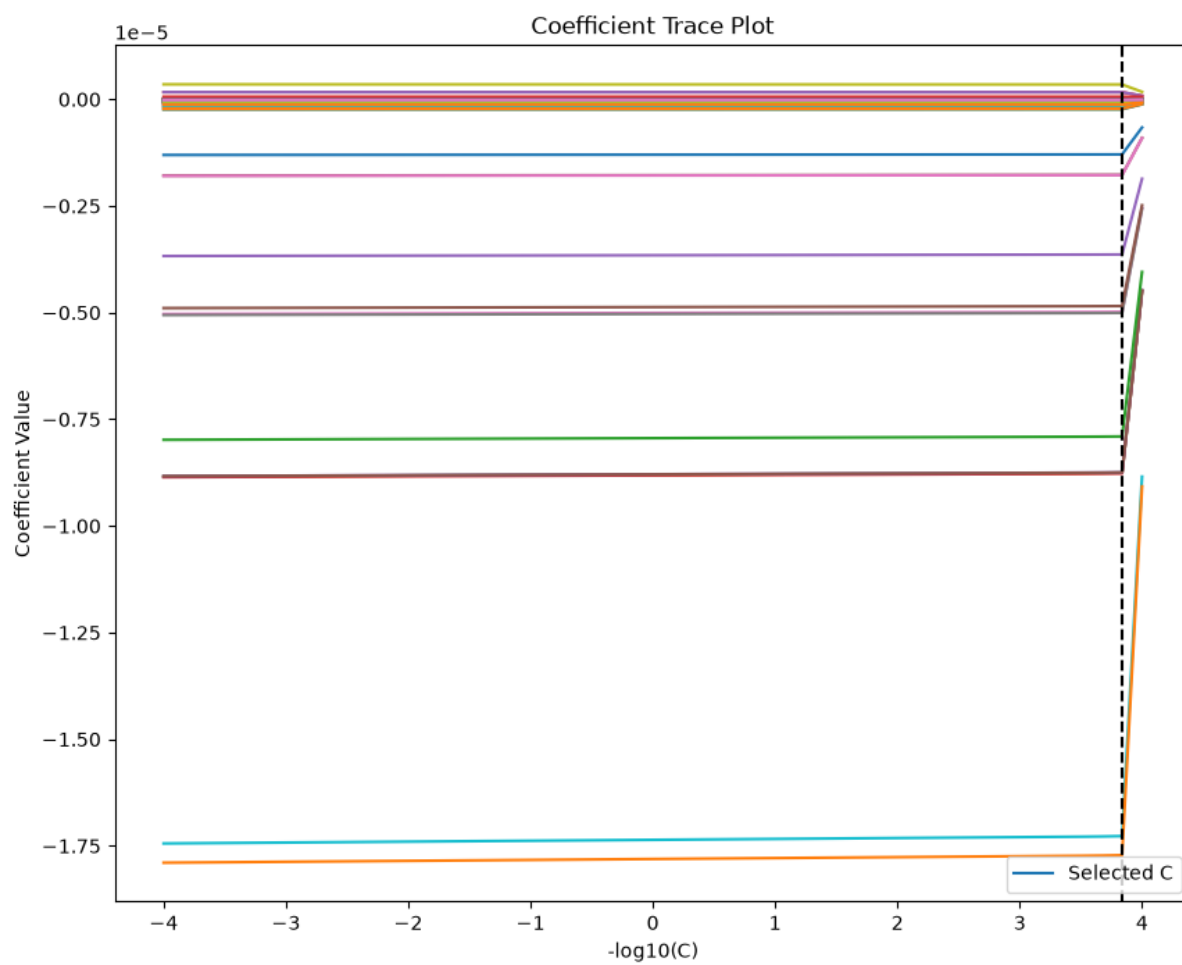


Figure 2: Elastic-Net regularisation paths across LOFO folds.

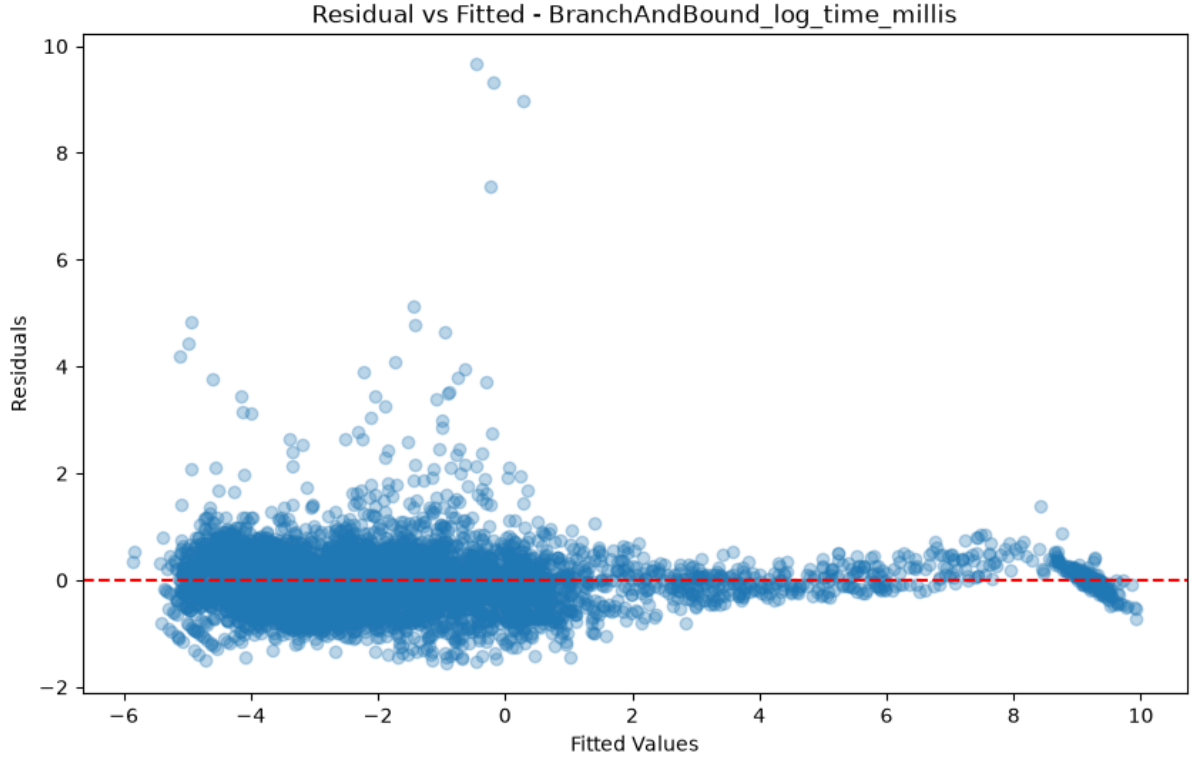


Figure 3: Residuals vs. fitted values for Branch and Bound `log_time_millis` ($M2$). Slight heteroskedasticity in the distribution tails is visible (Claim D04).

- [2] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.
- [3] Frank Hutter, Lin Xu, Holger H. Hoos, and Kevin Leyton-Brown. Algorithm runtime prediction: Methods & evaluation. *Artificial Intelligence*, 206:79–111, 2014.
- [4] Kevin Leyton-Brown, Eugene Nudelman, and Yoav Shoham. Empirical hardness models: Methodology and a case study on combinatorial auctions. *Journal of the ACM*, 56(4):1–52, 2014.
- [5] Leslie E. Papke and Jeffrey M. Wooldridge. Econometric methods for fractional response variables with an application to 401(k) plan participation rates. *Journal of Applied Econometrics*, 11(6):619–632, 1996.
- [6] David Pisinger. Where are the hard knapsack problems? *Computers & Operations Research*, 32(9):2271–2284, 2005.